

Exercise 8 — Unsupervised Learning

Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

Set 13 November 2026 · due Friday 20 November 2026, 23:59

Contents

What this exercise is for	1
The two account books	2
Part 1 — Predict, before you plot anything that could give it away (20 marks)	3
Part 2 — Measure (25 marks)	3
Part 3 — Explain the inversion (30 marks)	4
Part 4 — Mark yourself (25 marks)	4
Marking	5
Getting help	6

What this exercise is for

Lesson 8's own account table had one honest dimensionality — three latent factors behind eight observed columns — and one anomaly type: individually ordinary values that broke the relationship between columns. Principal component analysis (PCA) reconstruction error caught 37 of the 40 accounts flagged; a plain two-dimensional (2-D) scatter of the same accounts on their first two principal components would not have found them, because the anomalies sat, on average, **closer** to the middle of that picture than genuine accounts did (1.30 against 2.20), not farther — a “flag whatever sits at the edge” reading of that same plot points in exactly the wrong direction. It would be easy to leave the lecture believing that a 2-D PCA plot is simply the *wrong* tool for spotting an anomaly, full stop, and reconstruction error the *right* one, always.

That is not true either, and this exercise is built to show you when each one is.

You get **two account books from the same fictional bank, Meridian**, built the same way — the same two latent factors driving the same six features, the same 3% of accounts overwritten with a fraud pattern. On one book, a “look at the plot and flag whatever sits at the edge” detector does almost as well as reconstruction error. On the other, that same detector catches **not one single anomaly**, in ten independent draws of the data, while reconstruction error keeps working. Neither ordering can be guessed from the names of the two detectors; both can be predicted from twenty minutes spent reading how each book's fraud pattern was built, which is what makes this an unsupervised-learning exercise and not a coin flip.

That is the skill being assessed. Not fitting PCA — **reading how an anomaly was constructed well enough to know, before you plot anything, whether a picture will show it to you.**

Submit a single notebook, `exercise08_<surname>.ipynb`, that runs top to bottom in the course container. Written answers go in markdown cells beside the evidence for them.

The two account books

`account_monitoring_data.py` ships with this exercise and needs no network.

```
from account_monitoring_data import load_retail_accounts, load_business_accounts

retail = load_retail_accounts()      # personal current accounts
business = load_business_accounts()  # business current accounts
```

Each function returns a single DataFrame: the six features on file for every account, plus a boolean `is_anomaly` column. **`is_anomaly` is for scoring a detector after the fact, never for fitting one** — Meridian’s own fraud team does not have it, and neither should any PCA you fit below.

Six figures are on file for every account, retail or business: `monthly_inflow_eur`, `monthly_outflow_eur`, `transaction_count`, `avg_transaction_size_eur`, `night_transaction_ratio` and `new_payee_count`. Both books’ *genuine* accounts are generated by the same process: two latent factors, `cash_flow_scale` and `activity_level`, drive all six observed columns, plus each column’s own noise — so the honest dimensionality of either book is 2, not 6, and neither factor is ever shown to any method below.

Retail — a credential-stuffing takeover. An attacker has phished a login. They cannot move large sums immediately, so what shows up in the account’s activity for the week under review is several new payees added and probing at night, when a genuine retail customer rarely logs in. Everything about how much money moves through the account — inflow, outflow, average transaction size, overall transaction count — is left completely alone, still drawn from the same genuine process as every honest account. Only two of the six columns move, and they move a long way.

Business — a mule account. A shell company is opened purely to launder funds. Whoever set it up did not have a real business’s books to copy, so they picked each of the six numbers independently, one at a time, from roughly the range genuine business accounts occupy for that column. What they could not fake, because they did not know it existed, is *how the six numbers move together* in a genuine account — all six trace back to the same two latent factors, so they are correlated in a fixed way a one-column-at-a-time guess cannot reproduce.

The two books are the same size (1,600 accounts each) and the same contamination rate (3%), so neither sample size nor base rate is doing the work below — both are held fixed on purpose. What differs is **how the anomaly was built**, and that, not the name of any method, is what should drive your prediction in Part 1.

Read the module docstring in full before you do anything else — it describes both fraud patterns honestly and at length, and it is not padding. The `TRUE_*` constants below the loader functions record exactly how each book was built. **Do not read past the loader functions until Part 4.**

Both loaders take `n_samples`, `anomaly_fraction` and `random_state`; leave `n_samples` and `anomaly_fraction` at their defaults throughout, and vary only `random_state` where a part below asks you to.

Part 1 — Predict, before you plot anything that could give it away (20 marks)

1. Load both books and report, for each: the number of accounts, the number of planted anomalies, and the anomaly rate.
2. For each book, plot the six features split by `is_anomaly` — a pairwise scatter of a few feature combinations, or one strip/box plot per feature, whichever makes the anomalies easiest to see if they are visible at all. **Do not fit PCA or any detector yet.**
3. Then, in writing, **predict**: on which book would flagging “the points sitting farthest from the middle of a 2-D PCA plot” correctly catch most of the anomalies, and on which would it fail? Give a reason per book, in terms of lesson 8:
 - In the plots from step 2, do the anomalies stand out clearly on any single feature, or only — if at all — in how several features relate to each other?
 - Read the docstring’s description (not the `TRUE_*` numbers) of how each book’s anomalies were built. Does that construction sound like it would leave a mark on one column, or spread itself thinly across all six with no single column exposing it?
 - If an anomaly’s values are each individually unremarkable, would you expect its distance from the middle of a 2-D summary of *all six* columns to be unusually large, unusually small, or unremarkable?

What is being marked: the reasoning, not the ranking. A confident wrong prediction that engages with all three questions above scores well; a hedged answer that commits to nothing does not. **Write the prediction down before you run anything** — the rest of the exercise is much less instructive if you do not.

Part 2 — Measure (25 marks)

4. Standardise each book’s six features. Fit PCA on the **full, unlabelled** data (never pass `is_anomaly` to anything that fits) and report the scree table — explained variance ratio per component and its cumulative sum — for each book. State, with a reason tied to the table, how many components you would keep for each book.
5. Using the number of components you chose, build **two detectors** per book, both flagging the 3% of accounts that look most suspicious (3% is Meridian’s own estimate of its contamination rate — an estimate, not a certainty, the same way the handout’s section 6.1 flagged its own top 2%):
 - **“Look at the plot.”** Project the standardised data onto its first 2 principal components and flag the accounts farthest from the origin (PCA centres the data before projecting, so distance from the origin *is* distance from the data’s own centroid).
 - **Reconstruction error.** Using your chosen number of components, reconstruct every account and flag the accounts with the largest squared reconstruction error, exactly as section 6.1 does for Aurora’s table.
6. Report precision and recall for both detectors, on both books.
7. A single run of a synthetic generator tells you nothing about whether an effect is real or a lucky draw of 1,600 accounts — lesson 5’s point, applied here because there is no *y* to cross-validate against. Repeat steps 4–6 at 5 further seeds of each loader (vary `random_state` only)

and report each detector’s recall as a **mean and standard deviation** across all 6 runs, per book.

8. State plainly, book by book: do the two detectors agree?

What is being marked: that no detector or its fitted PCA ever sees `is_anomaly` except to be scored afterwards; that the number of components is chosen from the scree table and the choice is stated, not guessed; and that recall is reported with a spread across seeds, not read off one run.

One warning, worth a sentence in Part 3: if your business-book “look at the plot” recall comes out at **exactly 0.000** — precision 0.000 too, every single flagged account a false alarm — that is not a bug in your code. Check it against a second seed before you conclude anything, and then explain it; do not assume you have miscounted and quietly move on.

Part 3 — Explain the inversion (30 marks)

The ranking is not the same at the two books. Explain why, in six to ten sentences per book.

Support each explanation with a **measurement**, not an assertion. Suggestions, though you may find better ones:

- for a claim that a book’s anomalies do or do not stand out on any single column: for each book, compute the fraction of anomalous accounts whose value falls outside the genuine accounts’ 5th–95th percentile range, on **at least one** of the six columns;
- for a claim about where anomalies land in the 2-D projection: report the mean distance from the origin, anomalies against genuine accounts, per book — and say explicitly which direction the gap runs, since the direction is the entire point here, not just its size;
- for a claim about the reconstruction-error gap: report the mean reconstruction error, anomalies against genuine accounts, per book, at your chosen number of components;
- for a claim about *why* the number of kept components matters: rerun your reconstruction-error detector at 1, 2, 3, 4 and 5 kept components (both books, one fixed seed) and report recall at each. Aurora’s account table in the handout never had to confront a book where *more* components made detection worse — if you find one here, say at which component count it happens and why, in terms of what that extra component is doing on that particular book.

What is being marked: this part carries the most marks in the exercise, and the measurements are what separate an explanation from a plausible story. Lesson 8’s handout makes several claims of exactly this kind and backs every one with a number computed a second way, or at a second seed — do the same.

Part 4 — Mark yourself (25 marks)

Now open `account_monitoring_data.py` and read the `TRUE_*` constants and the two loader functions’ bodies.

9. `TRUE_N_LATENT_FACTORS` gives the number of latent factors that actually generated both books. Compare it against the number of components you chose from each book’s scree table in Part 2 — did the elbow point you to the right number on both books, the wrong number on one of them, or was it ambiguous?

10. `TRUE_RETAIL_SPIKE_FEATURES` names the two columns a credential-stuffing takeover disturbs, and `TRUE_RETAIL_SPIKE_PARAMS` gives the mean and standard deviation each is redrawn from. Using the **genuine** retail accounts' own mean and standard deviation for those two columns (computed from the data, not read off any constant), how many genuine standard deviations from the genuine mean does a typical spiked value land at, for each of the two columns?
11. The business book's anomalies are built by drawing each of the six columns independently from the genuine accounts' 5th–95th percentile range for that column. Explain, in your own words and referring to your own Part 3 numbers, why this construction makes an anomalous account's distance from the centroid in a 2-D PCA projection *smaller* on average than a genuine account's, rather than larger. The handout's section 6.2 makes the same argument for Aurora's table; restate it for this one.
12. Having seen exactly how each book's anomalies were built: would keeping only 1 principal component ever be a defensible choice for a fraud team that has to monitor **both** books with a single pipeline? Use the component-count sweep you ran in Part 3 to answer with numbers, not just a yes or no.

What is being marked: question 12. Being able to look at how an anomaly was actually generated and say *why* a particular amount of dimensionality reduction helps it or hides it — rather than treating “more compression is safer” or “less compression is safer” as a rule that holds everywhere — is the whole content of this lesson's second half.

Marking

Part	Marks
1 — Predict before plotting	20
2 — Measure	25
3 — Explain the inversion	30
4 — Mark yourself	25
Total	100

Marks are lost for:

- fitting PCA, or choosing which accounts to flag, using `is_anomaly` anywhere before the scoring step (−15);
- reporting a single run's recall instead of the mean and spread across the 6 seeds Part 2 asks for (−10);
- choosing a number of components by eye with no reference to the scree table, or not stating which number was chosen (−10);
- asserting a claim in Part 3 with no supporting number from your own run (−5 per unsupported claim);
- a notebook that does not run top to bottom in the container (−10).

Marks are **not** lost for a wrong prediction in Part 1, nor for measuring and reporting a recall of exactly 0.000 on the business book, provided it is checked rather than assumed and then explained in Part 3. A wrong prediction you then investigated is worth more than a cautious one that hedged.

Getting help

Bring questions to the start of lesson 9, or email fabio.antonini.1969@gmail.com. If you are stuck on Part 3, section 6.2 of the handout does exactly this kind of measurement for Aurora's own account table — it claims the anomalies sit closer to the centre than genuine accounts do, on average, and then checks that claim at two further seeds of the generator rather than trusting the first run.